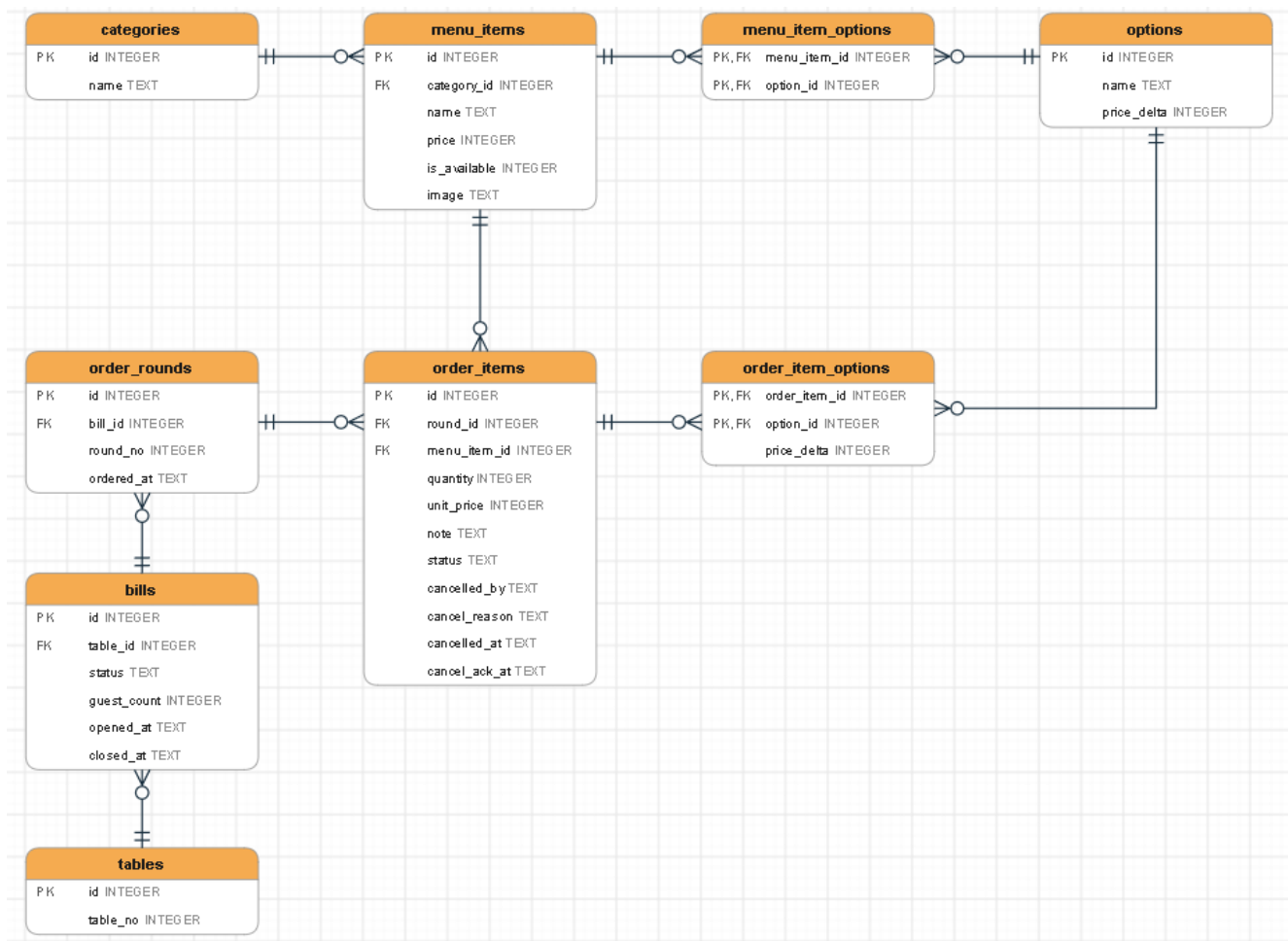


## 1. แผนผังฐานข้อมูล



## 2. เหตุผลของการแยกตาราง

เหตุผลหลักของการแยกตารางคือ ลดข้อมูลซ้ำ ป้องกันข้อมูลไม่สอดคล้องกัน

และทำให้การจัดการข้อมูลแต่ละส่วนทำได้ง่าย

หากนำข้อมูลทั้งหมดมารวมไว้ในตารางเดียว เช่น โต๊ะ บิล เมนู ราคา ตัวเลือก รอบ และจำนวน

จะเกิดการบันทึกข้อมูลโต๊ะ บิล และรายละเอียดเมนูซ้ำหลายครั้ง

สำหรับ Options ระบบใช้ menu\_item\_options เป็นตารางกลาง เนื่องจากเมนูหนึ่งสามารถมีหลาย Option และ Option หนึ่งสามารถใช้กับหลายเมนู จึงเป็นความสัมพันธ์แบบ Many-to-Many

- ลดข้อมูลซ้ำ

- แก้ไขข้อมูลได้ง่าย
- ลดความผิดพลาดจากข้อมูลไม่ตรงกัน
- รองรับการสัมพันธ์ระหว่างข้อมูลได้ชัดเจน
- สามารถขยายระบบในอนาคตได้ง่าย

### 3. การเก็บราคา

ระบบใช้วิธีเก็บราคาสำเนา ณ เวลาที่สั่งซื้อ หรือ Price Snapshot เพื่อรักษาราคาที่เกิดขึ้นจริงในขณะทำการ

```
order_items
unit_price INTEGER NOT NULL
```

```
order_item_options
price_delta INTEGER NOT NULL
```

สมมติว่าปัจจุบันเมนูกะเพราหมูมีราคา 60 บาท ลูกค้าสั่งในวันนี้ ระบบจะเก็บ unit\_price = 60 ไว้ใน order\_items หากวันถัดไปร้านเปลี่ยนราคาเป็น 70 บาท ข้อมูล menu\_items จะเปลี่ยนเป็น 70 แต่ Order เดิมยังคงมี unit\_price = 60

```
ตอนสั่ง
กะเพราหมู = 60 บาท
order_items.unit_price = 60
```

```
ภายหลังร้านเปลี่ยนราคา
menu_items.price = 70 บาท
```

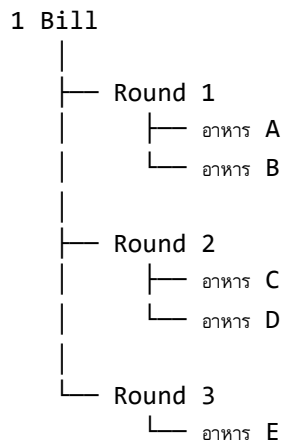
```
บิลเก่า
order_items.unit_price = 60 บาท
```

Option ก็ใช้หลักการเดียวกัน เช่น กะเพรา 60 บาท + ไข่ 10 บาท ระบบจะเก็บ unit\_price = 60 และ price\_delta = 10

เหตุผลที่เลือกวิธีนี้คือ ประวัติการขายต้องไม่เปลี่ยนตามราคาปัจจุบัน หากใช้ราคาจาก menu\_items โดยตรง บิลเก่าอาจถูกคำนวณด้วยราคาปัจจุบัน ซึ่งทำให้ข้อมูลทางการเงินไม่ถูกต้อง

## 4. การรองรับหลายรอบสั่ง

ระบบรองรับการสั่งอาหารหลายรอบภายในบิลเดียว โดยใช้โครงสร้าง bills → order\_rounds → order\_items



```
CREATE TABLE IF NOT EXISTS order_rounds (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  bill_id INTEGER NOT NULL,
  round_no INTEGER NOT NULL,
  ordered_at TEXT ...
)
```

เมื่อมีการสั่งรอบใหม่ ระบบจะหาเลขรอบล่าสุดของบิลแล้วเพิ่ม 1

```
SELECT ?, COALESCE(MAX(round_no), 0) + 1
FROM order_rounds
WHERE bill_id = ?
```

ดังนั้นถ้าบิลมี Round 1 และ Round 2 การสั่งครั้งต่อไปจะถูกบันทึกเป็น Round 3

การแยก order\_rounds ทำให้ระบบสามารถบอกได้ว่าอาหารแต่ละจานถูกสั่งในรอบใด และสามารถแสดงประวัติการสั่งได้อย่างชัดเจน

## 5. คำสั่ง SQL ที่ซับซ้อน

### SQL ที่ 1 – ค้นหาเมนู

```
SELECT m.id, m.category_id, c.name AS category_name,
       m.name, m.price, m.image, m.is_available
FROM menu_items m
JOIN categories c ON c.id = m.category_id
WHERE (? = '' OR instr(lower(m.name), lower(?)) > 0)
       AND (? = 0 OR m.category_id = ?)
```

```
AND (? = 0 OR m.is_available = 1)
ORDER BY m.category_id, m.id
```

คำสั่งนี้นำข้อมูลจาก menu\_items และ categories มา JOIN กัน แล้วกรองตามชื่อเมนู ประเภทอาหาร และสถานะการเปิดขาย

ผลลัพธ์คือสามารถค้นหาเมนูตามชื่อ เลือกหมวดหมู่ และเลือกแสดงเฉพาะเมนูที่เปิดขายได้

## SQL ที่ 2 – ส่ง Order

```
INSERT INTO order_rounds (bill_id, round_no)
SELECT ?, COALESCE(MAX(round_no), 0) + 1
FROM order_rounds
WHERE bill_id = ?

INSERT INTO order_items
(round_id, menu_item_id, quantity, unit_price, note)
SELECT ?, id, ?, price, ?
FROM menu_items
WHERE id = ? AND is_available = 1
```

ระบบจะสร้างรอบใหม่ก่อน จากนั้นเพิ่มรายการอาหารโดยอ่านราคาจากฐานข้อมูลโดยตรงและบันทึกเป็น unit\_price

การตรวจสอบ is\_available = 1 ช่วยป้องกันการสั่งเมนูที่ปิดขาย

## SQL ที่ 3 – คำนวณยอดบิล

```
SELECT COALESCE(SUM(line_total), 0) AS total
FROM order_lines
WHERE bill_id = ?
AND status <> 'cancelled'
```

คำสั่งนี้เลือกเฉพาะรายการของบิลที่กำหนด ไม่รวมรายการที่ถูกยกเลิก แล้วใช้ SUM() รวมยอด หากไม่มีรายการจะคืนค่า 0 ด้วย COALESCE()

## 6. Index และ EXPLAIN QUERY PLAN

ระบบมี Index ที่สำคัญสำหรับการค้นหา Order และรอบของบิล ดังนี้

**idx\_order\_items\_status**

```
CREATE INDEX IF NOT EXISTS idx_order_items_status ON order_items (status);
```

เหตุผล: ค้นหารายการอาหารตามสถานะ เช่น pending, cooking, ready

**idx\_order\_items\_round**

```
CREATE INDEX IF NOT EXISTS idx_order_items_round ON order_items (round_id);
```

เหตุผล: ค้นหารายการอาหารตามรอบ

idx\_order\_rounds\_bill\_round

```
CREATE UNIQUE INDEX IF NOT EXISTS idx_order_rounds_bill_round ON order_rounds (bill_id, round_no);
```

เหตุผล: ค้นหาของบิลและป้องกันเลขรอบซ้ำในบิลเดียวกัน

ผลจาก EXPLAIN QUERY PLAN

```
SEARCH r USING INDEX idx_order_rounds_bill_round (bill_id=?)  
SEARCH oi USING INDEX idx_order_items_round (round_id=?)
```

ผลลัพธ์แสดงว่า SQLite ใช้ idx\_order\_rounds\_bill\_round เพื่อค้นหา order\_rounds ตาม bill\_id และใช้ idx\_order\_items\_round เพื่อค้นหา order\_items ตาม round\_id

```
SEARCH oi USING INDEX idx_order_items_status (status=?)
```

สำหรับ Query ของหน้าครัว SQLite ใช้ idx\_order\_items\_status จริงในการค้นหารายการตามสถานะ แสดงว่า Index ที่สร้างมีประโยชน์ต่อ Query ที่ใช้งานจริง

## 7. ทรานแซกชัน (Transaction)

ระบบใช้ withTransactionAsync() ในการทำงานที่มีหลายคำสั่ง SQL ซึ่งต้องสำเร็จทั้งหมดหรือยกเลิกทั้งหมด เพื่อป้องกันข้อมูลค้างหรือข้อมูลไม่สมบูรณ์

### จุดที่ 1 – สร้างฐานข้อมูล

```
await db.withTransactionAsync(async () => {  
    await db.execAsync(SCHEMA_SQL);  
    await seedInitialData(db);  
    await db.execAsync("PRAGMA user_version = 1");  
});
```

การสร้างตาราง การ Seed ข้อมูล และการเปลี่ยน Version ควรสำเร็จทั้งหมด หากเกิดข้อผิดพลาดระหว่างทาง Transaction สามารถ Rollback ได้

ถ้าไม่ใช่ Transaction: หากการสร้างตารางสำเร็จ แต่การ Seed ข้อมูลหรือการกำหนด user\_version เกิดข้อผิดพลาด ระบบอาจอยู่ในสถานะสร้างฐานข้อมูลไปเพียงบางส่วน ทำให้โครงสร้างกับข้อมูลเริ่มต้นไม่สอดคล้องกัน และอาจทำให้แอปทำงานผิดพลาดในการเปิดใช้งานครั้งต่อไป.

การสร้างตาราง การ Seed ข้อมูล และการเปลี่ยน Version ควรสำเร็จทั้งหมด หากเกิดข้อผิดพลาดระหว่างทาง Transaction สามารถ Rollback ได้

## จุดที่ 2 – เพิ่มเมนู

การเพิ่มเมนูอาจประกอบด้วย的增加ข้อมูลใน menu\_items และการเพิ่มความสัมพันธ์ใน menu\_item\_options หากขั้นตอนใดผิดพลาด การใช้ Transaction ทำให้ไม่เกิดกรณีที่เมนูถูกสร้างแต่ Option ถูกบันทึกไม่ครบ

ถ้าไม่ใช้ Transaction: หากเพิ่มข้อมูลใน menu\_items สำเร็จ แต่การเพิ่มความสัมพันธ์ใน menu\_item\_options ล้มเหลว จะเกิดเมนูที่ถูกสร้างแล้วแต่ไม่มี Option ครบตามที่ผู้ใช้กำหนด ทำให้ข้อมูลเมนูไม่สมบูรณ์.

## จุดที่ 3 – แก้ไขเมนู

การแก้ไขเมนูอาจมีทั้ง UPDATE menu\_items, DELETE ความสัมพันธ์เดิม และ INSERT ความสัมพันธ์ใหม่ Transaction ช่วยให้การแก้ไขทั้งชุดเป็นการทำงานเดียวกัน

ถ้าไม่ใช้ Transaction: หาก UPDATE menu\_items สำเร็จ แล้วขั้นตอนลบหรือเพิ่ม menu\_item\_options เกิดข้อผิดพลาด เมนูอาจมีราคา/ชื่อใหม่ แต่ความสัมพันธ์ของ Option ยังเป็นข้อมูลเก่าหรือถูกลบไปบางส่วน ทำให้ข้อมูลไม่ตรงกัน.

## จุดที่ 4 – ส่ง Order

ตรวจสอบ Bill → สร้าง Order Round → สร้าง Order Item → สร้าง Order Item Option

หากรายการใดรายการหนึ่งผิดพลาด ระบบจะ Rollback

เพื่อไม่ให้เกิดกรณีที่บางรายการถูกบันทึกแต่บางรายการไม่ถูกบันทึก

## จุดที่ 5 – Reset Database

การ Reset ต้องลบข้อมูลหลายตารางแล้ว Seed ข้อมูลใหม่ จึงควรทำเป็น Transaction เดียว

เพื่อให้ฐานข้อมูลอยู่ในสถานะสมบูรณ์เสมอ

ถ้าไม่ใช้ Transaction: หากลบข้อมูลไปบางตารางแล้วเกิดข้อผิดพลาดก่อน Seed ข้อมูลใหม่เสร็จ ฐานข้อมูลอาจอยู่ในสภาพข้อมูลหายบางส่วน และไม่สามารถกลับมาอยู่ในสถานะเริ่มต้นที่สมบูรณ์ได้.

## 8. ปัญหาที่เจอและวิธีแก้

### ปัญหาที่ 1 — ราคาในบิลเก่าเปลี่ยนตามเมนูปัจจุบัน

**อาการที่เห็น:** เมื่อมีการเปลี่ยนราคาเมนูในภายหลัง หากระบบนำราคาปัจจุบันจาก menu\_items ไปคำนวณบิลเก่า ยอดของรายการเดิมอาจเปลี่ยนตามราคาใหม่ ทำให้ราคาที่แสดงในประวัติการขายไม่ตรงกับราคาที่ถูกจ่ายจริง

**สาเหตุที่แท้จริง:** รายการสั่งซื้ออ้างอิงเพียง menu\_item\_id แล้วดึง price ปัจจุบันจากตาราง menu\_items แทนที่จะเก็บราคาที่เกิดขึ้นจริง ณ เวลาที่สั่ง

**วิธีแก้:** เก็บราคาสำเนาไว้ใน order\_items.unit\_price และเก็บส่วนต่างราคาของตัวเลือกไว้ใน order\_item\_options.price\_delta ตั้งแต่ตอนสร้าง Order เพื่อให้ราคาของบิลเก่าคงเดิมแม้เมนูจะเปลี่ยนราคา

### ปัญหาที่ 2 — เมนูที่ปิดขายยังสามารถถูกสั่งได้

**อาการที่เห็น:**

ตอนปิดเมนูบางรายการไม่ให้ขายแล้ว แต่ผู้ใช้ที่เปิดหน้ารายการอาหารค้างไว้ก็ยังเห็นเมื่อนั้นอยู่ และสามารถกดเข้าไปเพิ่มในตะกร้าได้

**สาเหตุที่แท้จริง:**

หน้ารายการอาหารโหลดข้อมูลไว้ก่อน เมื่อพนักงานเปลี่ยน is\_available ของเมนูภายหลัง หน้าของลูกค้านั้นยังไม่ได้โหลดข้อมูลใหม่ ทำให้ข้อมูลที่แสดงไม่ตรงกับฐานข้อมูลปัจจุบัน

**วิธีแก้:**

ตอนส่ง Order ให้ตรวจสอบสถานะของเมนูจากฐานข้อมูลอีกครั้งก่อนบันทึก ถ้า is\_available = 0 ก็ไม่ให้บันทึกการรายนั้น ถึงแม้ว่าผู้ใช้จะยังเห็นเมนูอยู่ในหน้าจอก็ตาม

### ปัญหาที่ 3 — การส่ง Order อาจบันทึกข้อมูลไม่ครบ

**อาการที่เห็น:** การส่ง Order หนึ่งครั้งต้องบันทึกหลายส่วน เช่น รอบสั่ง รายการอาหาร และตัวเลือกอาหาร หากคำสั่งใดคำสั่งหนึ่งล้มเหลว อาจเกิดข้อมูลถูกบันทึกเพียงบางส่วน เช่น มีรอบสั่งแต่รายการอาหารไม่ครบ

**สาเหตุที่แท้จริง:** การบันทึก Order ประกอบด้วยหลายคำสั่ง SQL หากแต่ละคำสั่งทำงานแยกจากกัน เมื่อเกิด Error ระหว่างทาง คำสั่งก่อนหน้าสามารถถูกบันทึกไปแล้ว

**วิธีแก้:** ใช้ db.withTransactionAsync() ครอบกระบวนการส่ง Order ทั้งหมด เมื่อทุกคำสั่งสำเร็จจึง Commit แต่ถ้ามีข้อผิดพลาดจะ Rollback กลับไปก่อนเริ่มทำการ

## 9. การแบ่งงาน

ตัวอย่างรูปแบบการแบ่งงาน สามารถปรับเปอร์เซ็นต์ให้ตรงกับการทำงานจริงของสมาชิก

| สมาชิก                                | ส่วนงาน                                                                                                           |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| นาย พนรัตน์ ประสิทธิพันธุ์ 6721651572 | ฝั่งลูกค้า หน้าเมนู ค้นหาและกรอง, ตะกร้า, ส่งออเดอร์หลายรอบ, สรุปบิล, ปิดบิล                                      |
| นาย ญาณวุฒิ ภิระบรรณ 6721651254       | ฐานข้อมูล โต๊ะ และจัดการหน้าร้าน<br>Schema.sql, ข้อมูลตั้งต้น, ปุ่มล้างหน้า, หน้าเลือกและเปิดโต๊ะ, หน้าจัดการเมนู |
| นาย วีรชิต แต่งแก้ว 6721651807        | ฝั่งครัวและโครงแอป หน้าครัว, สถานะรายการ, ยกเลิกและแจ้งเตือน, navigation                                          |

## 10. สิ่งที่จะทำต่อถ้ามีเวลา

**เพิ่มระบบประวัติการแก้ไขราคา:** บันทึกว่าใครเปลี่ยนราคา เปลี่ยนจากเท่าไรเป็นเท่าไร และเปลี่ยนเมื่อใด

**เพิ่มระบบรายงานยอดขาย:** เช่น ยอดขายรายวัน เมนูขายดี จำนวนอาหารที่ขาย ยอดขายแต่ละโต๊ะ และยอดขายแต่ละช่วงเวลา

**เพิ่มการทดสอบกรณีผิดพลาด:** เช่น สั่งเมนูที่ปิดขาย เลือก Option ที่ใช้กับเมนูไม่ได้ ปิดบิลขณะอาหารยังไม่เสร็จ และยกเลิกอาหาร